

OpenSeek-3 Collatz Conjecture 技术报告

1. 任务概述

本任务对应 OpenSeek-3 的 **Collatz Conjecture** 子任务。任务要求对输入整数列表中的每一个整数执行一次 Collatz 变换，并输出变换后的整数列表。

Collatz 变换规则如下：

- 如果当前整数是偶数，则将其除以 2；
- 如果当前整数是奇数，则将其乘以 3 后再加 1。

也就是说，对于任意整数 x ，其转换结果为：

```
1 | if x % 2 == 0:
2 |     new_x = x // 2
3 | else:
4 |     new_x = 3 * x + 1
```

例如，输入为：

```
1 | [72, 29, 49]
```

逐个元素执行转换：

```
1 | 72 是偶数，因此 72 / 2 = 36
2 | 29 是奇数，因此 29 * 3 + 1 = 88
3 | 49 是奇数，因此 49 * 3 + 1 = 148
```

最终输出为：

```
1 | [36, 88, 148]
```

需要注意的是，本任务并不是要求持续执行 Collatz 序列直到到达 1，而是只对列表中的每个元素执行 **一步 Collatz 变换**。代码实现中也是按照这一点进行设计的。

2. 数据集结构与输入输出格式

本任务的数据文件为 JSON 格式。代码通过 `load_task(file_path)` 函数读取数据，并从 JSON 文件中解析出任务信息、训练样例和测试样例。

数据文件主要包含以下字段：

1	task_id
2	task_name
3	Definition
4	examples
5	test_samples

各字段含义如下：

字段	说明
task_id	当前任务编号，例如 <code>openseek-3</code>
task_name	当前任务名称，例如 <code>Collatz Conjecture</code>
Definition	任务自然语言定义，说明 Collatz 转换规则
examples	训练样例，包含输入和标准输出
test_samples	测试样例，只包含输入，需要生成预测结果

训练样例通常具有如下结构：

1	{
2	"__id__": "sample_id",
3	"input": "[72, 29, 49]",
4	"output": "[36, 88, 148]"
5	}

测试样例通常具有如下结构：

1	{
2	"__id__": "test_sample_id",
3	"input": "[72, 29, 49]"
4	}

程序最终会生成 JSONL 格式的提交文件，每一行对应一个测试样例：

1	{ "test_sample_id": "openseek-3-xxxx", "prediction": "[36, 88, 148]" }
---	--

其中：

- `test_sample_id` 来自测试样例中的 `__id__` 字段；
- `prediction` 是程序计算出的转换后列表；
- `prediction` 需要以字符串形式保存。

3. 总体设计思路

本代码采用“模型生成求解器 + 自动验证 + 失败回退 + 批量预测 + 结果保存”的整体流程。

与直接手写固定求解函数不同，本代码首先根据任务定义和样例构造提示词，让 Qwen3-4B 模型生成一个符合要求的 Python `solve(input_text: str) -> str` 函数。之后程序会对生成函数进行编译和验证。如果模型生成的函数在训练样例上能够达到要求，则使用该函数处理测试集；如果模型生成失败或验证效果不理想，则回退到内置的保底求解器。

整体流程如下：

```
1  读取任务 JSON 文件
2      ↓
3  获取 task_id / task_name / Definition / examples / test_samples
4      ↓
5  构造代码生成 Prompt
6      ↓
7  调用 Qwen3-4B 生成 solve 函数代码
8      ↓
9  提取 Python 代码块
10     ↓
11 使用 exec 编译生成代码
12     ↓
13 在 examples 上验证生成 solver 的准确率
14     ↓
15 若准确率为 100%，使用生成 solver
16 若多次尝试仍失败，则回退到 fallback solver
17     ↓
18 对训练集进行完整评估
19     ↓
20 对测试集进行预测
21     ↓
22 保存完整 JSON 结果文件
23     ↓
24 保存比赛提交 JSONL 文件
```

该设计的核心目的是提高自动化程度。模型负责根据任务描述生成求解代码，程序负责对生成代码进行验证、筛选和执行，避免直接信任模型输出。

4. 核心算法原理

本任务的核心算法非常明确：对输入列表中的每个整数执行一次 Collatz 转换。

假设输入列表为：

```
1  nums = [x1, x2, x3, ..., xn]
```

输出列表为：

```
1  result = []
```

对每个元素 `x` 执行：

```
1 if x % 2 == 0:
2     result.append(x // 2)
3 else:
4     result.append(x * 3 + 1)
```

最终返回：

```
1 str(result)
```

4.1 偶数处理

当 `x` 是偶数时，使用整除：

```
1 x // 2
```

例如：

```
1 72 -> 36
2 10 -> 5
3 4 -> 2
```

这里使用 `//` 而不是 `/`，是因为输出需要保持整数。如果使用 `/`，Python 会返回浮点数，例如 `72 / 2 = 36.0`，这会导致输出格式和标准答案不一致。

4.2 奇数处理

当 `x` 是奇数时，执行：

```
1 x * 3 + 1
```

例如：

```
1 29 -> 88
2 49 -> 148
3 7 -> 22
```

4.3 复杂度分析

设输入列表长度为 `n`。

程序只需要遍历列表一次，因此时间复杂度为：

```
1 O(n)
```

程序需要保存输出列表，因此空间复杂度为：

该算法是本任务的最优解之一，因为必须至少读取每个输入元素一次。

5. 数据加载模块

代码中的数据加载函数如下：

```
1 def load_task(file_path: str) -> Tuple[str, str, List[str], List[Dict[str, Any]],  
  List[Dict[str, Any]]]:  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4     return (  
5         data.get("task_id", ""),  
6         data.get("task_name", ""),  
7         data.get("Definition", []),  
8         data.get("examples", []),  
9         data.get("test_samples", []),  
10    )
```

该函数完成以下工作：

1. 打开本地 JSON 数据文件；
2. 使用 `json.load` 解析 JSON 内容；
3. 读取任务 ID；
4. 读取任务名称；
5. 读取任务定义；
6. 读取训练样例；
7. 读取测试样例；
8. 对缺失字段提供默认值。

该函数返回五个对象：

```
1 task_id, task_name, definition, examples, test_samples
```

主程序后续的所有流程都依赖这些数据，包括 prompt 构造、solver 生成、solver 验证、测试集预测和结果保存。

6. 模型 API 调用模块

代码使用 OpenAI SDK 创建了一个兼容 OpenAI 接口格式的客户端：

```

1 client = OpenAI(
2     api_key="dummy",
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1",
4 )

```

模型调用函数为：

```

1 def qwen_api(messages: List[Dict[str, str]], model: str = "/Qwen3-4B/Qwen/Qwen3-4B",
2   retries: int = 3) -> str:
3     for attempt in range(retries):
4         try:
5             res = client.chat.completions.create(
6                 model=model,
7                 messages=messages,
8                 temperature=0.0,
9             )
10            return res.choices[0].message.content or ""
11        except Exception as e:
12            if attempt == retries - 1:
13                print(f"\nAPI 调用失败: {e}")
14                return ""
15            time.sleep(2)
16    return ""

```

该函数具有以下特点。

6.1 使用 Qwen3-4B 模型

代码默认调用：

```

1 /Qwen3-4B/Qwen/Qwen3-4B

```

该模型用于根据任务说明和样例自动生成 Python 求解函数。

6.2 使用低温度生成

代码设置：

```

1 temperature=0.0

```

这表示尽量减少模型输出随机性。对于代码生成任务而言，稳定性比创造性更重要，因此 `temperature=0.0` 是合理设置。

6.3 支持重试机制

函数设置：

```

1 retries=3

```

如果模型接口调用失败，程序最多尝试三次。前两次失败后等待 2 秒继续重试；如果最后一次仍失败，则打印错误信息并返回空字符串。

这种设计可以应对临时网络波动或服务端响应失败，增强程序运行稳定性。

7. Python 代码块提取模块

模型生成代码时，有时会直接返回纯代码，有时会返回 Markdown 代码块，例如：

```
1  ```python
2  def solve(input_text: str) -> str:
3      ...
4  ```
```

为了兼容这两种情况，代码定义了正则表达式：

```
1  CODE_BLOCK_PATTERN = re.compile(
2      r"```python\s*(.*?)```|```\s*(.*?)```",
3      re.DOTALL | re.IGNORECASE
4  )
```

并定义代码提取函数：

```
1  def extract_code(text: str) -> str:
2      if not text:
3          return ""
4      match = CODE_BLOCK_PATTERN.search(text)
5      if match:
6          return (match.group(1) or match.group(2) or "").strip()
7      return text.strip()
```

该函数逻辑如下：

1. 如果模型输出为空，直接返回空字符串；
2. 如果模型输出中包含 ```python 代码块，则提取代码块内容；
3. 如果模型输出中包含普通 ``` 代码块，也提取其中内容；
4. 如果没有代码块，则将完整文本作为代码返回。

这一步可以提高模型输出解析的容错性，使程序不依赖模型完全按照纯代码格式输出。

8. 保底求解器设计

代码定义了 `fallback_solver_code()` 函数，用于在模型生成代码失败时提供内置求解器。

保底代码如下：

```

1 def fallback_solver_code() -> str:
2     return ''
3 def solve(input_text: str) -> str:
4     nums = ast.literal_eval(input_text.strip())
5     result = []
6     for x in nums:
7         if x % 2 == 0:
8             result.append(x // 2)
9         else:
10            result.append(x * 3 + 1)
11    return str(result)
12    ''.strip()

```

该保底求解器的逻辑非常直接：

1. 使用 `ast.literal_eval` 将输入字符串解析为 Python 列表；
2. 创建空列表 `result`；
3. 遍历输入列表中的每个整数；
4. 如果整数为偶数，则加入 `x // 2`；
5. 如果整数为奇数，则加入 `x * 3 + 1`；
6. 返回 `str(result)`。

8.1 fallback 的意义

模型生成代码存在一定不稳定性，可能出现以下问题：

- 生成结果不是完整 Python 代码；
- 没有定义 `solve` 函数；
- 输出格式不符合要求；
- 逻辑错误；
- 编译失败；
- 运行时报错；
- 在训练样例上准确率不足。

因此，fallback 求解器可以作为最后保障，确保即使模型生成失败，程序仍然可以继续完成测试集预测。

8.2 当前代码中的潜在问题

当前 `fallback_solver_code()` 中使用了：

```
1 ast.literal_eval(input_text.strip())
```

但是 `compile_solver` 函数执行代码时使用的是：

```
1 namespace: Dict[str, Any] = {}
2 exec(code, namespace)
```

如果 fallback 代码本身没有写入 `import ast`，并且 `namespace` 中也没有提前放入 `ast`，那么 fallback 的 `solve` 函数在实际调用时可能会出现：

```
1 NameError: name 'ast' is not defined
```

因此，更稳妥的做法有两种。

第一种是在 fallback 代码中加入：

```
1 import ast
```

第二种是在 `compile_solver` 中提供 `ast`：

```
1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {"ast": ast}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]
```

从代码可靠性角度看，第二种方式更统一，因为模型生成代码和 fallback 代码都可以共享外部注入的 `ast` 对象。

9. Prompt 构造模块

代码通过 `build_code_generation_prompt` 函数构造用于代码生成的用户提示词：

```
1 def build_code_generation_prompt(task_name: str, definition: List[str], examples:
2   List[Dict[str, Any]]) -> str:
3     demo_examples = examples[:10]
4     examples_text = "\n".join(f"输入: {ex['input']}\n输出: {ex['output']}" for ex
5   in demo_examples)
6     return f'''
7     你是 Python 算法工程师。
8     请根据任务定义和样例生成可执行 Python 代码。
9
10    任务名: {task_name}
11    任务定义:
12    {chr(10).join(definition)}
13
14    样例:
15    {examples_text}
16
17    严格要求:
18    1. 定义函数 solve(input_text: str) -> str
```

```
17 2. input_text 是完整字符串，例如 "[1,2,3]"
18 3. 可以使用 import 或标准库
19 4. 用 ast.literal_eval 解析输入
20 5. 返回值必须是字符串
21 6. 生成代码必须能被 exec 直接执行
22 '''
```

该提示词包含以下信息：

1. 模型角色：Python 算法工程师；
2. 任务名称；
3. 任务定义；
4. 前 10 条训练样例；
5. 严格输出要求。

9.1 使用样例进行上下文学习

代码选择：

```
1 demo_examples = examples[:10]
```

也就是说，使用前 10 条 examples 作为示例，帮助模型理解输入输出格式和转换规则。

对于本任务来说，规则较简单，10 条样例通常足以让模型归纳出：输入是整数列表字符串，输出是执行一次 Collatz 变换后的列表字符串。

9.2 明确 solve 函数接口

提示词要求生成代码必须定义：

```
1 solve(input_text: str) -> str
```

这一接口是后续自动编译、验证和批量预测的基础。如果模型没有生成该函数，程序就无法统一调用。

9.3 允许使用标准库

提示词允许：

```
1 可以使用 import 或标准库
```

这使模型可以生成 `import ast` 等语句，从而正确解析输入字符串。但与此同时，这也意味着安全限制较弱，具体问题会在后续“安全性与风险分析”中说明。

10. 编译 solver 模块

代码通过 `compile_solver` 函数将模型生成的代码字符串转换为可调用的 Python 函数：

```

1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]

```

该函数执行流程如下：

1. 创建空命名空间 `namespace`；
2. 使用 `exec(code, namespace)` 执行生成代码；
3. 检查命名空间中是否存在 `solve`；
4. 如果不存在，则抛出异常；
5. 如果存在，则返回 `solve` 函数对象。

10.1 动态执行的作用

动态执行使程序可以将模型生成的文本代码变成真正可调用的函数。这样后续验证和预测阶段就可以统一调用：

```

1 solve_func(sample["input"])

```

10.2 动态执行的风险

当前代码中写明“取消安全限制”，并直接执行模型生成代码：

```

1 exec(code, namespace)

```

这在受控比赛环境中可能可以接受，但从工程安全角度看存在风险。如果模型生成了文件操作、系统命令、网络请求等代码，就可能造成安全问题。

更安全的做法包括：

- 禁止危险 import；
- 限制 `__builtins__`；
- 使用 AST 分析检查代码结构；
- 只允许定义 `solve` 函数；
- 禁止 `open`、`exec`、`eval`、`compile`、`os`、`subprocess` 等危险操作；
- 在隔离环境中执行生成代码。

11. solver 验证模块

代码使用 `validate_solver` 函数验证生成的求解器是否正确：

```

1 def validate_solver(solve_func: Callable[[str], str], examples: List[Dict[str, Any]],
2   limit: int = 100) -> Tuple[float, List[Dict[str, Any]]]:
3     total = min(limit, len(examples))
4     errors: List[Dict[str, Any]] = []
5     correct = 0
6     for sample in examples[:total]:
7         try:
8             pred = str(solve_func(sample["input"])).strip()
9         except Exception as e:
10            pred = f"<runtime_error: {e}>"
11        gt = str(sample["output"][0]).strip()
12        if pred == gt:
13            correct += 1
14        else:
15            errors.append({"id": sample.get("id", ""), "input": sample["input"], "gt":
16                          gt, "pred": pred})
17    return correct / total if total else 0.0, errors

```

该函数的主要功能是：

1. 从 examples 中取前 `limit` 条样例；
2. 调用 `solve_func` 生成预测；
3. 与标准答案进行字符串比较；
4. 统计正确数量；
5. 收集错误样例；
6. 返回准确率和错误列表。

11.1 字符串级比较

代码使用：

```

1 pred = str(solve_func(sample["input"])).strip()
2 gt = str(sample["output"][0]).strip()

```

然后判断：

```

1 pred == gt

```

这意味着输出格式必须严格一致。例如：

```

1 [36, 88, 148]

```

和：

```

1 [36,88,148]

```

在字符串层面并不相等。虽然两者表达的列表内容相同，但如果标准答案包含空格，而预测结果没有空格，就可能被判为错误。

当前 fallback 使用 `str(result)`，Python 列表默认格式会包含逗号和空格，通常与数据集输出格式一致。因此该做法在本任务中是合理的。

11.2 运行时异常处理

如果 `solve_func` 在某条样例上运行出错，程序不会中断，而是将预测记录为：

```
1 <runtime_error: ...>
```

并将该样例加入错误列表。这有助于后续分析错误原因。

12. 自动生成 solver 模块

代码中的核心生成函数为 `generate_valid_solver`：

```
1 def generate_valid_solver(task_name: str, definition: List[str], examples:
  List[Dict[str, Any]], max_attempts: int = 3):
2     last_code = ""
3     last_errors = []
4     for attempt in range(1, max_attempts + 1):
5         ...
```

该函数最多尝试 3 次生成 solver。

12.1 第一次生成

第一次尝试时，程序调用：

```
1 prompt = build_code_generation_prompt(task_name, definition, examples)
```

该 prompt 包含任务定义和前 10 条样例。模型根据这些信息生成完整 Python 代码。

消息格式如下：

```
1 messages = [
2     {"role": "system", "content": "你是严格输出可执行 Python 代码的助手。"},
3     {"role": "user", "content": prompt}
4 ]
```

之后程序调用：

```
1 raw_code = qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B")
2 code = extract_code(raw_code)
```

提取出 Python 代码。

12.2 编译与验证

生成代码提取后，程序尝试编译：

```
1 solve_func = compile_solver(code)
```

然后在训练样例上验证：

```
1 acc, errors = validate_solver(solve_func, examples, limit=min(200, len(examples)))
```

如果准确率达到 100%，立即返回：

```
1 return code, solve_func, acc, errors
```

12.3 基于错误样例的再次生成

如果第一次生成的函数没有达到 100% 准确率，程序会记录错误样例，并在下一次 prompt 中加入错误反馈：

```
1 error_text = "\n".join(  
2     f"输入: {e['input']} | 正确输出: {e['gt']} | 你的输出: {e['pred']}"  
3     for e in last_errors[:10]  
4 )
```

新的 prompt 会告诉模型：

```
1 你上一次生成的 solve 函数在以下样例上出错：  
2 ...  
3 请重新生成完整 Python 代码，并修正逻辑。
```

这种设计相当于一个简单的自我修正流程。模型不是每次从零开始生成，而是根据上一轮错误案例修复逻辑。

12.4 回退机制

如果多次尝试后仍然失败，程序输出：

```
1 回退到保底求解器。
```

然后使用：

```
1 last_code = fallback_solver_code()  
2 solve_func = compile_solver(last_code)
```

最后再次验证 fallback solver，并返回结果。

13. 预测模块

代码使用 `run_predictions` 函数对训练集或测试集执行预测：

```
1 def run_predictions(solve_func: Callable[[str], str], samples: List[Dict[str, Any]],
2   has_label: bool=False):
3     results = []
4     correct = 0
5     for s in tqdm(samples, desc="Running solver"):
6         try:
7             pred = str(solve_func(s["input"])).strip()
8         except Exception as e:
9             pred = f"<runtime_error: {e}>"
10            item = {"test_sample_id": s.get("id", ""), "prediction": pred}
11            if has_label:
12                gt = str(s["output"][0]).strip()
13                item["gt"] = gt
14                item["is_correct"] = pred == gt
15                if item["is_correct"]:
16                    correct += 1
17            results.append(item)
18            output = {"total": len(samples), "details": results}
19            if has_label:
20                output["correct"] = correct
21                output["accuracy"] = correct / len(samples) if samples else 0.0
22            return output
```

该函数可以同时用于训练集和测试集。

13.1 用于训练集

当 `has_label=True` 时，函数会读取标准答案，并计算：

```
1 item["is_correct"] = pred == gt
```

最终返回：

```
1 {
2     "total": 样例总数,
3     "details": 每条样例预测详情,
4     "correct": 正确数量,
5     "accuracy": 准确率
6 }
```

13.2 用于测试集

当 `has_label=False` 时，函数只保存：

```

1  {
2      "test_sample_id": 样例 ID,
3      "prediction": 预测结果
4  }

```

不会计算准确率。

13.3 使用 tqdm 显示进度

函数使用：

```

1  tqdm(samples, desc="Running solver")

```

显示处理进度，方便运行时观察程序是否正常推进。

14. 主流程分析

主程序位于：

```

1  if __name__ == "__main__":

```

其中定义了三个路径：

```

1  file_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
   Annotation\data\openseek-3_collatz_conjecture.json"
2  out_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
   Annotation\rgs_q3\experiment\openseek3_collatz_results.json"
3  test_jsonl_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
   Annotation\experiment\openseek-3-v1.jsonl"

```

三个路径分别表示：

路径变量	作用
<code>file_path</code>	输入数据集 JSON 文件路径
<code>out_path</code>	保存完整评估结果的 JSON 文件路径
<code>test_jsonl_path</code>	保存最终比赛提交文件的 JSONL 路径

14.1 加载数据

主流程首先执行：

```

1  task_id, task_name, definition, examples, test_samples = load_task(file_path)

```

然后打印任务信息：

```
1 print(f"任务: {task_id} / {task_name}")
2 print(f"训练样例数: {len(examples)}, 测试样例数: {len(test_samples)}")
```

14.2 生成 solver

随后调用：

```
1 generated_code, solve_func, preview_acc, preview_errors = generate_valid_solver(
2     task_name, definition, examples, max_attempts=3
3 )
```

该步骤会自动完成代码生成、编译、验证和 fallback。

14.3 训练集完整评估

程序对 examples 进行完整评估：

```
1 example_eval = run_predictions(solve_func, examples, has_label=True)
```

这一步可以得到完整训练集准确率。

14.4 测试集预测

程序对 test_samples 进行预测：

```
1 test_eval = run_predictions(solve_func, test_samples, has_label=False)
```

得到每条测试样例的预测结果。

14.5 保存完整 JSON 结果

程序将生成代码、预验证准确率、错误样例、训练集评估结果和测试集预测结果保存到 `out_path`：

```
1 output_data = {
2     "task_id": task_id,
3     "task_name": task_name,
4     "evaluate_time": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),
5     "generated_code": generated_code,
6     "preview_accuracy_on_examples": preview_acc,
7     "preview_errors": preview_errors[:20],
8     "examples_eval": example_eval,
9     "test_eval": test_eval
10 }
```

该 JSON 文件适合用于后续复盘和调试。

14.6 保存 JSONL 提交文件

最后，程序将测试集预测结果保存为 JSONL：

```
1 os.makedirs(os.path.dirname(test_jsonl_path), exist_ok=True)
2 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
3     for item in test_eval["details"]:
4         line = json.dumps(item, ensure_ascii=False)
5         f.write(line + '\n')
```

该文件可直接用于比赛提交。

15. 输出文件说明

本代码会生成两个主要输出文件。

15.1 完整结果 JSON 文件

路径由 `out_path` 指定。该文件包含完整运行信息，例如：

```
1 {
2   "task_id": "openseek-3",
3   "task_name": "Collatz Conjecture",
4   "evaluate_time": "2026-xx-xx xx:xx:xx",
5   "generated_code": "...",
6   "preview_accuracy_on_examples": 1.0,
7   "preview_errors": [],
8   "examples_eval": {
9     "total": 100,
10    "correct": 100,
11    "accuracy": 1.0,
12    "details": []
13  },
14  "test_eval": {
15    "total": 100,
16    "details": []
17  }
18 }
```

该文件主要用于实验记录和调试。

15.2 JSONL 提交文件

路径由 `test_jsonl_path` 指定。每一行是一个测试样例预测结果，例如：

```
1 {"test_sample_id": "openseek-3-xxxx", "prediction": "[36, 88, 148]"}
```

该文件是最终提交给评测系统的结果文件。

